CERA Tariff Comparison Tool — Technical Architecture & Security Review

Document purpose: technical reference for an expert/external reviewer (functional + security assessment) prior to production go-live. Originally written 2026-07-14; updated 2026-07-15 to reflect the `SECURITY_REMEDIATION_CHECKLIST.md` hardening pass, a follow-on security review covering SQL injection, buffer overflow, broken access control, email injection, OTP brute-force, dependency CVEs, and JWT storage (see §13 and §14), and a later same-day pass covering accidental-overload protection, sliding session expiry, and two cross-portal access-control gaps found during manual testing (see §15). Updated again 2026-07-23 (see §16) to reflect five days of subsequent functional/data-integrity work: a supplier↔user foreign key replacing string matching, a scheduled local database backup, and an append-only-changelog bug fix.

Scope: full-stack review of `tariff-api/` (FastAPI backend) and the three static frontend applications (`admin.html`, `supplier.html`, `customer.html`) plus supporting scripts.

---

1. System overview

A tariff comparison and regulatory-monitoring tool for the Cyprus Energy Regulatory Authority (CERA). Three audiences, three portals, one backend:

Portal	File	Auth	Purpose
Customer calculator	`customer.html`	None (public)	Compare electricity tariffs from approved suppliers
Supplier portal	`supplier.html`	JWT + OTP 2FA	Suppliers submit/edit their own tariffs, upload supporting documents, read CERA broadcast messages
Admin panel	`admin.html`	JWT + OTP 2FA	CERA staff approve/reject tariffs, manage suppliers/users/regulated charges/DAM data, send broadcast messages

Stack

- Backend: Python 3.14, FastAPI, SQLAlchemy 2.0 (sync ORM), served by `uvicorn`
- Database: PostgreSQL 17 (migrated from SQLite on 2026-07-10; SQLite remains as a fallback if `DATABASE_URL` is unset — used by the IT-review test package)
- Auth: PyJWT (HS256), `bcrypt` for password hashing (direct `bcrypt` library, not `passlib`)
- Email: Microsoft Graph API via `msal` (OAuth2 client-credentials flow), no SMTP/passwords
- Frontend: no build step, no framework — three self-contained static HTML files with inline CSS and vanilla JS, served by FastAPI's `StaticFiles` mount
- Process model: single uvicorn process; an `asyncio` background task inside the same process handles scheduled notifications and password-expiry checks (no separate worker/queue)

Repository layout (relevant files)

```
tariff-api/
  main.py            FastAPI app, CORS, static mount, startup schema migrations, background scheduler
  auth.py            JWT issuance/verification, password hashing/policy, lockout-aware get_current_user
  database.py        SQLAlchemy engine/session (Postgres via DATABASE_URL, SQLite fallback)
  models.py          ORM models (16 tables)
  schemas.py         Pydantic request schemas
  serializers.py      ORM → dict serializers shared by public/admin endpoints
  importer.py        Bulk config import (legacy "Publish" flow)
  email_utils.py     Graph API email templates and sending
  routers/auth.py    /api/auth/* — login, OTP, password reset/change
  routers/admin.py   /api/admin/* — all authenticated CRUD + messaging + scheduling
  create_user.py     CLI: create/update/deactivate a user account
  unlock_user.py     CLI: clear an account lockout
  seed.py            CLI: bulk-load a legacy JSON config into the DB
  scripts/           One-off migration SQL (supplier FK backfill/cleanup) + backup_db.ps1 (§16.2)
  uploads/           Uploaded PDF documents (server-side, uuid-named)
  .env               Local secrets (see §5 — currently plaintext; excluded from version control via .gitignore)
admin.html / supplier.html / customer.html / reset-password.html   Static frontend pages
start.bat            Dev launcher: creates venv, installs deps, runs uvicorn
```

Version control: a local git repository was initialized 2026-07-14 (previously there was none — flagged as a process gap in the original version of this document). `.gitignore` excludes `.env`, `tariff-api/.venv/`, `tariff-api/uploads/*` (except a `.gitkeep`), and `*.db` files. There is no remote configured — history exists locally only, which is adequate for a single-developer project but worth noting if collaboration or off-machine backup of history becomes a requirement.

---

2. Request flow

1. Browser loads `admin.html` / `supplier.html` / `customer.html` directly from FastAPI's static mount (same origin as the API — no separate frontend server).
2. `customer.html` calls only `GET /api/public/config` (unauthenticated) and renders everything client-side from that single JSON blob (suppliers, profiles, approved tariffs, regulated charges, DAM price data, notices, changelog).
3. `admin.html` / `supplier.html` show a login overlay first. Successful login issues either a full JWT or a short-lived "pending" OTP token (see §4). **As of 2026-07-15**, the full JWT is delivered via an `httpOnly` / `Secure` / `SameSite=Strict` session cookie set by the server (`Set-Cookie`), not returned in the JSON response body — the browser attaches it automatically to same-origin requests, and JavaScript cannot read it at all (see §14.7). Because the client can no longer synchronously know whether it's authenticated (the cookie isn't JS-readable), each portal's page-load logic optimistically calls its first authenticated data endpoint and lets a 401 response (rather than a locally-stored flag) trigger the login overlay.
4. All API routers are registered **before** the catch-all static mount in `main.py`, so `/api/...` paths always take priority over static file serving.

---

3. Data model

16 tables, all in `models.py`. Key groupings:

- **Reference data**: `suppliers`, `profiles` (half-hourly load-shape curves parsed from EAC CSV exports)
- **Core business object**: `tariffs` — one row per tariff, `rates` / `discounts` stored as JSON text columns (frontend calculation logic consumes them as plain JSON; no need to normalize further at this scale). Lifecycle status: `draft → pending → approved/rejected`, plus `suspended` (admin can pull a live tariff without deleting it).
- **Regulated charges**: `regulated` (current values, single row) + `regulated_history` (append-only snapshot on every change, for trend history)
- **Market data**: `dam_data` — single-row table holding Day-Ahead Market price data as JSON arrays (monthly, half-hourly-by-slot, and raw daily)
- **Content**: `notices` (bilingual EN/EL rich text shown in the calculator)
- **Audit**: `changelog` (field-level diffs on every tariff/regulated-charge change, records actor role+email), `login_activity` (every login attempt, success or failure, with IP/user-agent)
- **Auth/accounts**: `users`, `password_history` (last 5 hashes, reuse prevention), `password_resets` (1-hour single-use tokens, also reused for the supplier invite-link flow), `otp_codes` (10-minute single-use 6-digit codes). **Updated 2026-07-20 (§16.1)**: `User.supplier_id` is now a real FK to `suppliers.id` (was a plain `supplier_name` string that had to exactly match `Supplier.name`, with no enforced relationship); `MessageRead` got the same FK treatment. `Tariff.supplier` and `Message` / `ScheduledNotification.target_suppliers` remain plain strings.
- **Documents**: `tariff_documents` — uploaded PDF metadata; the file itself lives in `uploads/<uuid>.pdf`, decoupled from the user-supplied filename
- **Messaging**: `messages`, `message_reads` (read + acknowledge tracking per supplier), `scheduled_notifications` (recurring broadcast rules, e.g. "remind suppliers every 6 months")

Schema evolution is handled by a hand-rolled migration block at the top of `main.py` (`ALTER TABLE ... ADD COLUMN` wrapped in try/except with per-statement rollback for Postgres) rather than Alembic. This works but has no down-migration path and no version tracking — acceptable for the current single-developer/single-environment stage, but worth automating (Alembic) before the schema needs to move across multiple environments (dev/staging/prod) in lockstep.

---

4. Authentication & authorization

Login flow

1. `POST /api/auth/login` — email+password checked against `bcrypt` hash.
2. **Account lockout**: 3 failed attempts locks the account for 180 minutes; lockout and unlock are both logged to `login_activity`, and a lockout triggers an email alert to the account owner (`notify_account_locked`). Unknown emails and known-but-wrong-password both return generic

401s, but only known accounts increment `failed_attempts` (avoids leaking which emails exist while still supporting lockout).

3. On success, if the user has `otp_enabled` (default true), the server does **not** issue a real JWT — it emails a 6-digit OTP (10-minute expiry, one active code per email, 2-minute resend cooldown) and returns a short-lived "pending" token (`purpose: "otp"` claim) that `get_current_user` explicitly rejects for any real API call.
4. `POST /api/auth/verify-otp` exchanges the pending token + code for a real JWT (sliding expiry — see Token design below).
5. `otp_enabled` can be toggled off per-user by an admin (documented as intended for testing) — this is a legitimate operational escape hatch but is also a control an external reviewer will want to see is restricted to admins and audited (it is admin-only via `require_admin` , but toggling it is **not** currently written to `changelog` or any audit trail — only `login_activity.reason` would show `password_only` afterwards).

Token design

- HS256 JWT, claims: `sub` (user id), `email` , `role` , `supplier` , `iat` , `exp` , `session_start` .
- **Expiry, updated 2026-07-15 (§15.2): sliding, not flat.** Originally a flat 8-hour expiry from login. Now `TOKEN_EXPIRE_HOURS = 1` is a *sliding* window — `get_current_user` reissues the cookie with a fresh 1-hour expiry on every successful authenticated request — capped by `ABSOLUTE_SESSION_HOURS = 12` measured from the original login via the `session_start` claim, which is preserved unchanged across renewals (only `iat` / `exp` advance). This means an active user is never logged out mid-work purely because a fixed clock ran out, while a stolen/replayed token still can't be kept alive indefinitely.
- **Delivery, updated 2026-07-15**: the JWT is set via `Set-Cookie` (httpOnly/Secure/SameSite=Strict, no `Max-Age` — a browser-session cookie, so it doesn't outlive the browser session regardless of the server-side expiry) rather than returned in the JSON body. `get_current_user` reads it from `request.cookies` . A `POST /api/auth/logout` endpoint clears the cookie server-side (`Set-Cookie` with an expired date) — necessary because JavaScript cannot clear an httpOnly cookie itself.
- **Identity re-hydration, added 2026-07-15 (§15.3)**: `GET /api/auth/me` returns `{role, supplier, email}` for the current session. Added because the frontend previously only knew the logged-in user's identity from the login response itself, cached in `sessionStorage` — a valid cookie in a *new* tab or after a reload had no way to repopulate the user badge / Sign-out control. Both portals now call this endpoint to re-hydrate identity whenever `sessionStorage` is empty but the session cookie is still valid.
- **Session invalidation on password change**: `get_current_user` compares the token's `iat` against `user.password_changed_at` and rejects tokens issued before the most recent password change — closes the window where a stolen/old token stays valid after a reset.
- **Password-expiry gate**: once `must_change_password` is set (see below), every endpoint except `PUT /api/auth/password` returns 403 `password_expired` until the user sets a new password.

Password policy

- Minimum 8 chars, must include a letter, a digit, and a symbol (`validate_password_strength`).

- Reuse prevention: cannot reuse the current password or any of the last 5 (`password_history` , checked via `bcrypt.checkpw` against each stored hash — correctly avoids a plaintext comparison table).
- **180-day expiry**: a daily background check (`run_password_expiry_check` , driven by the in-process scheduler in `main.py`) flags overdue passwords and emails a one-time reminder 14 days before expiry.

Authorization model

Two roles only: `admin` (CERA staff) and `supplier` (scoped to one `supplier_name`). Enforcement pattern is consistent throughout `routers/admin.py` : - `get_current_user` — any authenticated user (used for GET endpoints and supplier-writable resources) - `require_admin` — admin-only (all supplier/regulated/notices/DAM/user-management/messaging writes) - `_assert_tariff_owner` — explicit ownership check on every tariff mutation so a supplier token can never touch another supplier's tariff, even by guessing a tariff ID - Tariff-document endpoints repeat the same ownership check independently (upload/list/download) rather than sharing a single dependency — functionally correct today but a maintenance risk if a new document endpoint is added without copying the check.

CSRF, updated 2026-07-15: the session now lives in a cookie (see §2, §14.7), which does auto-attach to requests — the control that makes classic CSRF not apply is `SameSite=Strict` on that cookie, not "bearer-token-only" as in the original design. `SameSite=Strict` means the browser never sends the cookie on a request originating from another site, including top-level navigations, so a malicious third-party page cannot ride the victim's session even to a same-origin endpoint. Combined with the app having no cross-origin form actions anywhere and CSP's `form-action 'self'` (see §8), this is considered sufficient without a separate CSRF-token scheme for this app's architecture (single origin, no third-party embedding, no public cross-site API consumers). CORS is restricted to `TARIFF_BASE_URL` (see §6), which is the correct control for cross-origin fetch/XHR attempts specifically.

---

5. Secrets & configuration

All configuration is environment-variable driven (`tariff-api/.env` , loaded via `python-dotenv`). Current variables:

- `AZURE_TENANT_ID` , `AZURE_CLIENT_ID` , `AZURE_CLIENT_SECRET` — Azure AD app registration for Graph API email sending
- `TARIFF_EMAIL_FROM` , `TARIFF_ADMIN_EMAIL` — mailbox identities
- `TARIFF_BASE_URL` — public base URL (used for CORS origin, email links, and OTP/reset links)
- `TARIFF_SECRET_KEY` — JWT signing secret
- `DATABASE_URL` — Postgres connection string (includes DB password)

⚠️ **Open finding (already tracked internally, not yet remediated):** `.env` holds all of the above in plaintext on disk. This is the single most obvious finding an external reviewer will surface. It must be replaced with a proper secret store (Azure Key Vault, or environment variables injected by the hosting platform at deploy time) before production deployment or external inspection. The project folder was

already moved off OneDrive (2026-07-10) to remove cloud-sync exposure, but the file itself is still not access-controlled beyond OS file permissions.

⚠️ **Related finding:** the Azure app registration's `Mail.Send` permission is an **application permission** ("send mail as any user" — confirmed against Microsoft's own permission description in the Azure Portal), not delegated. The app is correctly restricted to "My organization only" (rules out cross-tenant abuse), but nothing yet scopes *which* mailboxes within the tenant it can impersonate — that requires an Exchange Online **Application Access Policy**, which needs a Global/Exchange admin to apply (the operating account does not have the required role). This should be raised as a distinct action item from the `.env` plaintext issue, not folded into it.

`auth.py` has a hardcoded fallback JWT secret (`dev-only-do-not-deploy-...`) used only if `TARIFF_SECRET_KEY` is unset — clearly labeled as dev-only in an adjacent comment, and superseded by a real generated value in the current `.env` . Confirm before go-live that the environment used for external inspection/production actually has `TARIFF_SECRET_KEY` set (the fallback exists precisely so an operator can't accidentally deploy with no secret configured, but it can't stop them deploying with the *same known* fallback if the env var is missing).

---

6. Network/transport controls

- **CORS**: `allow_origins=[TARIFF_BASE_URL]` (was previously `"*"` , tightened 2026-07-10). Verified live: unlisted origins receive no CORS header; the configured origin does. `allow_methods` / `allow_headers` remain `"*"` , which is fine given the origin restriction.
- **HTTPS/TLS**: not handled by the app itself — `uvicorn` is run directly on port 8000 with no TLS. This is acceptable for local dev but **must** sit behind a reverse proxy (nginx/Caddy/Azure App Service's built-in TLS termination) with HTTPS before any external exposure. Currently there is no reverse proxy in front of it.
- `--reload` **must never be used** in this environment: with Python 3.14 / uvicorn 0.49, `--reload` causes HTTP 500 on all POST endpoints. `start.bat` and all documented run instructions correctly omit it.
- **Rate limiting, added 2026-07-15 (§15.1)**: a per-IP sliding-window limiter (60 requests/minute) guards against *accidental* overload — a stuck frontend retry loop or a mis-run script — rather than sophisticated abuse. Self-contained middleware in `main.py` , not a third-party library; see §15.1 for why.

---

7. File uploads

`POST /api/admin/tariffs/{id}/documents` accepts supplier-uploaded supporting PDFs: - Extension check (`.pdf`) **and** magic-byte check (`content.startswith(b"%PDF")`) — resistant to simple extension-renaming bypass. - 10 MB size cap enforced after reading the full body into memory (acceptable at this file-size ceiling; would need streaming for much larger limits). - Stored server-side under a `uuid4().hex + ".pdf"` filename — the original filename is preserved only as metadata (`original_name`) and never used to construct a filesystem path, which avoids path-traversal via crafted filenames. - Download endpoint re-

checks tariff ownership for supplier tokens before serving the file (`GET /api/admin/documents/{doc_id}/download`). - **Fixed 2026-07-15**: the download endpoint used to build its `Content-Disposition` header via an unescaped f-string embedding the uploaded file's original filename — a `"` in that filename could inject extra header directives. Fixed by removing the manual header override and relying on Starlette's built-in `FileResponse(filename=...)` handling, which runs the name through `urllib.parse.quote()` and automatically switches to RFC 5987 `filename*=utf-8''...` encoding whenever the quoted form would differ. - Uploading a new document on an approved/rejected tariff automatically reverts it to `pending` — correct business logic (an admin approval covers a specific document set; a new document should trigger re-review), and is itself changelog-worthy — currently this state transition is **not** written to `changelog` (only status changes made via the explicit status-patch endpoint are). Minor audit-trail gap.

No antivirus/content scanning of uploaded PDFs — acceptable given admin-approval gates every tariff, but worth noting for the external reviewer's threat model if these PDFs are later served to end customers rather than just CERA staff.

---

8. Frontend architecture & client-side findings

All three portal pages are single-file monoliths (no bundler, no framework) — `admin.html` (~5,360 lines / ~166 functions), `supplier.html` (~1,890 lines / ~74 functions), `customer.html` (~2,560 lines / ~77 functions), plus a small standalone `reset-password.html` (~150 lines).

Auth handling (admin/supplier), updated 2026-07-15: the session token was originally stored in `sessionStorage` and sent explicitly via `Authorization: Bearer` . It has since been migrated to an httpOnly cookie (see §4, §14.7) specifically because `sessionStorage` is JS-readable — any successful XSS (present or future) could exfiltrate it with `sessionStorage.getItem(...)` . An httpOnly cookie cannot be read by JavaScript at all, closing that theft vector even if a new XSS bug is found later. Every portal still consistently redirects to the login overlay on any 401, now driven by the first authenticated data call's response rather than a locally-stored flag (see §2).

customer.html is confirmed public/read-only: the entire file makes exactly one network call, `fetch('/api/public/config')` (GET, unauthenticated), and uses no `localStorage` / `sessionStorage` at all.

✅ XSS findings — exhaustively reviewed and fixed 2026-07-14

An initial sampled review found one instance in `customer.html` . Live exploit testing then kept turning up more instances the sample had missed, so the review was escalated to a full manual audit of **every single one of the 116 `innerHTML` / `outerHTML` / `insertAdjacentHTML` call sites** across all four frontend files (76 in `admin.html` , 19 in `supplier.html` , 20 in `customer.html` , 1 in `reset-password.html`) — not a sample. All findings below have been remediated and re-verified; kept here as the record and as the model to follow for any new render code.

Root cause: all three portals are vanilla-JS monoliths (no framework, no templating layer), so every dynamic render is a hand-built string assigned to `.innerHTML` . Any database-sourced string interpolated

into one of these without an escaping helper is a stored-XSS sink. A second, sneakier variant recurred repeatedly: embedding such a string inside an inline `onclick="fn('${x}')"` handler is unsafe **even when HTML-escaped**, because the browser HTML-decodes the attribute value before parsing it as JS — an escaped `'` (`'`) decodes back to a literal `'` and still breaks the JS string. The only real fix is to stop embedding the value in the handler at all (`data-*` attribute + `this.dataset`, or an id-keyed lookup cache).

Findings, roughly in order of severity:

1. **Unauthenticated stored XSS via the login-activity audit table** (`admin.html`) — the single worst finding. `POST /api/auth/login` logs the submitted `email` and the request's `User-Agent` header verbatim into `login_activity`, with **no authentication and no account required** — anyone can hit the public login endpoint with an arbitrary string in either field. Both were rendered unescaped in the admin-only login-activity table. This is a stored-XSS path that needs zero valid credentials of any kind and lands directly in an admin's authenticated session.
2. **Dashboard pending-review widget** (`admin.html`, `buildDashboard()`) — renders every pending tariff's name/supplier and fires **automatically the instant an admin logs in**, no click required.
3. **Tariff review modal** (`admin.html`, `openReview()`) — where an admin inspects a supplier's pending submission before approving it: `t.supplier`, `t.name`, `t.contract` / `voltage` / `pricing`, `prof.name`, discount labels, and changelog diff values (`ch.from` / `ch.to`) were all unescaped. A malicious submission could execute script directly in the admin's session during approval review.
4. **Public customer.html** — the original finding: `tariff.name` / `tariff.supplier` (one case inside an inline `onclick`), profile names, and supplier contact-card fields (address/phone/email/website/notes) unescaped on the fully public, unauthenticated calculator page.
5. **`supplier.html`'s `openDocsModal(id, name)`** — embedded the tariff name inside an inline-`onclick` JS string; worse than the admin.html equivalents because `supplier.html`'s own pre-existing `esc()` didn't escape single quotes at all, so a bare `'` in a tariff name broke it immediately, no entity-decoding subtlety required.
6. **Document-download `onclick` in both `admin.html` and `supplier.html`** — built as `onclick="downloadDoc(id,'${original_name.replace(/'/g,"\\'")}')"`; escaping only the JS single quote does not protect this context, so a `"` or `<` in an uploaded PDF's filename could still break attribute/JS context.
7. **Changelog table + its supplier filter dropdown, a second supplier-filter dropdown on the tariff-management tab, the "group by supplier" header, the supplier-management list, the profiles list (`name` / `desc`), the edit-tariff and invite-user supplier dropdowns, the audit-calculator panel, and tier-rate `value=` input attributes in both `admin.html` and `supplier.html`** — same unescaped-interpolation pattern, lower individual severity but same fix.
8. **`admin.html`'s user-management table** — `user.email` / `user.supplier_name` unescaped in the table and inside several inline `onclick` handlers. Lower severity since these are admin-set at account creation.

Fix applied uniformly: a consistent `esc()` HTML-escaping helper (escaping `& < > " '`) was added to `admin.html` and `customer.html`, and the pre-existing `esc()` in `supplier.html` was upgraded to also escape `'`. Every finding above now routes through it. Every inline-`onclick`-embedding case was additionally restructured to pass only an id and resolve the associated string from a small lookup

(`_docsCache` , `_usersCache` , `tariffs.find(...)` , or a `data-*` attribute) rather than ever embedding a database-sourced string inside event-handler JS source.

Verification — three escalating rounds, the last two genuine live exploit tests against the real app: 1. Static check: started the dev server, loaded both main pages via Playwright, confirmed zero console/page errors, confirmed `esc()` neutralizes representative payloads in isolation. 2. Live end-to-end test: seeded a **disposable SQLite database** (the real Postgres dev DB was never touched — `DATABASE_URL` overridden per-process to a scratchpad temp file, Azure email credentials blanked so no real email could send) with a malicious supplier and tariff record, ran the real FastAPI app against it on a spare port, and drove real Chromium via Playwright through the actual public-calculator and admin-review-modal flows. This caught the dashboard widget and a filter dropdown that the first fix pass had missed. 3. After escalating to the full 116-sink manual review (which surfaced the login-activity, profiles-list, and `supplier.html` `openDocsModal` findings above): re-seeded the test database with payloads targeting every newly-fixed site — a malicious profile name/description, a tariff renamed to a bare single-quote payload, an **unauthenticated** login POST with a malicious email, and a malicious `User-Agent` header — and re-ran the full live test across both an admin session and a supplier session. **Final result: 0 script executions across 7 distinct payload markers, 0 dialogs, 0 page errors.**

Separate, also-fixed finding — `javascript:` / `data:` URL scheme injection: HTML-escaping (`esc()`) does not protect against a malicious URL *scheme*, since a payload like `javascript:alert(1)` contains none of the characters escaping touches. `customer.html` 's two supplier-contact-card renderers built `href="${esc(s.website)}"` from the supplier's self-entered website field — a malicious/compromised supplier could have set their "website" to a `javascript:` URL that executes when any customer clicks the contact link on the public page. **Fix:** added a `safeUrl()` helper that only allows `http://` / `https://` through (anything else, including bare domains with no scheme, becomes `#`); verified live that `javascript:` , mixed-case `JaVaScRiPt:` , and `data:` payloads are all neutralized while a real `https://` URL passes through unchanged. `admin.html` / `supplier.html` don't render `website` as a clickable link (admin.html shows it as plain text), so no equivalent gap existed there. `tel:` / `mailto:` links elsewhere are safe by construction — the scheme prefix is a hardcoded template literal, not user-controlled.

No hardcoded secrets or API keys were found in any frontend file.

Content-Security-Policy, added 2026-07-14 (was previously absent — see §13.2): a `Content-Security-Policy` response header is now set via FastAPI middleware on every response (not via `<meta>` tag). `script-src` / `style-src` retain `'unsafe-inline'` (the frontend's ~hundreds of inline `onclick` handlers and inline `<script>` blocks would need a full refactor to `addEventListener` to remove it, not currently scheduled) plus the two CDN hosts actually used (cdnjs for Chart.js/DOMPurify, Google Fonts). Everything else is locked down: `object-src 'none'` , `base-uri 'self'` , `form-action 'self'` , `frame-ancestors 'self'` , `connect-src 'self'` , `img-src 'self' data:` . Because of the `'unsafe-inline'` allowance, this CSP does **not** block the class of bug the escaping/DOMPurify fixes address — it guards against loading external malicious scripts/styles and locks down framing, forms, plugins, and cross-origin connections, as defense-in-depth on top of (not instead of) output escaping.

Three follow-up items from this pass, all closed 2026-07-14 (see §13.1, §13.4):

- ~~The admin notices rich-text editor intentionally renders raw HTML (a WYSIWYG feature, by design) — meaning a compromised admin account has a direct stored-XSS path to the public page.~~ **Closed** via `SECURITY_REMEDIATION_CHECKLIST.md` item 1: DOMPurify 3.1.6 now sanitizes at all 4 sinks touching

this content (`customer.html` 's tab renderer and vulnerable-customer modal; `admin.html` 's editor display and save-back path), so a compromised admin account can no longer inject executable script via the notices editor.

- **CSV formula injection**: `admin.html` 's changelog CSV export interpolated tariff/supplier names into CSV cells unescaped; a name starting with `=` , `+` , `-` , or `@` could be interpreted as a formula by Excel/Sheets on open. Fixed with a `csvCell()` helper that prefixes a leading `'` on any such value and correctly doubles embedded `"` per RFC4180 (a pre-existing quoting bug found alongside the injection fix).
- **DOM-based XSS via `location.hash` / `location.search` / `postMessage`** : grepped all 4 frontend files for this pass — the only hit was `reset-password.html` 's `token` / `portal` query-string parsing, confirmed safe (never rendered via `innerHTML` , only compared as a string or forwarded as opaque data). No `postMessage` listeners exist anywhere in the app.

---

9. Email / notification subsystem

`email_utils.py` sends all mail via Microsoft Graph (`POST /v1.0/users/{from}/sendMail`), authenticated with `msal.ConfidentialClientApplication` client-credentials flow — no SMTP credentials anywhere. All sends are fire-and-forget on a daemon thread (`_send_bg`) so a slow/failing Graph call never blocks the request that triggered it; failures are logged, never raised to the caller. Templates are table-based HTML with inline styles for Outlook/Gmail compatibility. Notification triggers: new tariff submission, resubmission, approval, rejection, password reset, account lockout, OTP code, password-expiry warning, supplier invite, and broadcast messages (manual and scheduled). Bilingual (EN/EL) templates are used for the password-expiry and broadcast-message notifications; the rest are English-only.

---

10. Background scheduler

A single `asyncio` task started on FastAPI startup (`_scheduled_notifications_loop` , `main.py`) runs every hour in-process: fires any due recurring `scheduled_notifications` , and once per day also runs the password-expiry check. This is explicitly a **single-instance design** — the code comment notes that scaling to multiple app instances would double-send without adding a lock. Fine for the current Basic B1 App Service / single-process deployment target; must be revisited if the app is ever scaled horizontally.

---

13. Post-checklist hardening (2026-07-14) — `SECURITY_REMEDIATION_CHECKLIST.md`

Four forward-looking hardening items identified during the original review, all closed 2026-07-14 (full detail lives in `SECURITY_REMEDIATION_CHECKLIST.md` ; summarized here for the reviewer):

13.1 DOMPurify for the notices rich-text path. The admin notices editor (assumptions/legal/vulnerable-customer text, EN/EL) intentionally renders raw HTML — a WYSIWYG feature, not a bug, but it meant a compromised admin account had a direct stored-XSS path to the public page (see §8). DOMPurify 3.1.6 (vendored via cdnjs, the same CDN already used for Chart.js) now sanitizes at all 4 sinks: `customer.html`'s `switchNoticesTab()` / `openVulnerableModal()` and `admin.html`'s `setNotices()` / `getNotices()`, so a payload can't round-trip through the admin's own editor either.

13.2 Content-Security-Policy header. See §8 for the policy detail and the `'unsafe-inline'` tradeoff.

13.3 Server-side input validation (`tariff-api/schemas.py`). Rewrote the Pydantic schemas: `Literal[...]` types for every enum-like field (`contract`, `voltage`, `pricing`, `fpaSource`, `loadEntitlement`, tariff `status`, message/notification `audience`) matching the exact value sets the frontend's `<select>` elements use — a raw API call with a valid JWT can no longer set an arbitrary string in these fields. Added `max_length` bounds to every free-text field and `ge` / `le` bounds on numeric duration fields. `SupplierIn.email` / `CreateUserIn.email` now validate through the `email-validator` library; `SupplierIn.website` now requires an `http://` / `https://` scheme — this is the root-cause fix for the `javascript:`-URL finding in §8 (a supplier can no longer *submit* such a URL, not just have it neutralized at render time by `safeUrl()`). `rates` / `discounts` remain loosely-typed JSON since their shape varies by pricing type.

13.4 Reduced `innerHTML` reliance at the 3 highest-risk sinks. Rewrote `admin.html`'s `renderLoginActivity()`, `buildDashboard()`'s pending-review widget, and the tariff review modal (`openReview()`) to build real DOM nodes (`document.createElement` / `.textContent`) instead of HTML template-literal strings — untrusted data can only ever render as text at these 3 sinks now, independent of whether `esc()` is called correctly. This was a targeted rewrite of the most severe sinks, not all ~116 `innerHTML` sites; the remainder still rely on the `esc()` escaping pattern documented in §8, which was verified live in the original XSS pass.

Also closed alongside this pass: the CSV formula-injection fix and the DOM-based-XSS (`location` / `postMessage`) grep documented in §8.

---

14. Post-XSS security review (2026-07-15)

A further pass covering vulnerability classes not addressed by the original review or the checklist above: SQL injection, buffer overflow, broken access control, email injection, OTP brute-force, dependency CVEs, and JWT storage. All 7 items closed; each was verified live against a disposable SQLite database (the real Postgres dev database was never touched) using the project's actual `.venv` Python environment.

14.1 SQL injection — checked, clean. Every data query in `tariff-api/` goes through SQLAlchemy's ORM (`db.query(Model).filter(Model.col == value)`), which parameterizes automatically. The only raw SQL in the codebase is `main.py`'s startup migration block (hardcoded `ALTER TABLE` literals, one bound `:now` parameter) — no user input reaches it.

14.2 Buffer overflow — not an applicable vulnerability class. Pure Python/JS stack, no `ctypes` / `struct` /native extensions handling raw buffers. The PDF-upload path (the only raw-bytes

handling in the app) uses Starlette's dynamically-sized `UploadFile.read()` with the size cap, magic-byte check, and UUID storage filename already described in §7.

Bonus finding, fixed alongside this item: the document-download `Content-Disposition` header injection described in §7.

14.3 Broken access control / IDOR — found and fixed. Three routes in `routers/admin.py` did not enforce the ownership checks used consistently elsewhere (§4):

- `GET /api/admin/config` and `GET /api/admin/changelog` returned the **entire changelog unfiltered** to any authenticated supplier — other suppliers' tariff names, diffs, and admin actor emails, even though the tariffs list itself was correctly scoped. Fixed by filtering `Changelog.supplier == user.supplier_name` for supplier-role users.
- `GET /api/admin/suppliers` returned every supplier's full contact record (address/email/tel/website/licence) to any logged-in supplier, despite the file's own docstring already saying "admin only". Fixed by switching its dependency to `require_admin` — confirmed via grep that no frontend page needed the unfiltered version (`admin.html` gets suppliers through the already-scoped `/config` bulk endpoint).

Verified live with two distinct supplier accounts, each with their own tariff and changelog entry — confirmed one supplier's token can no longer see the other's data, while an admin token still sees everything. 8/8 checks passed.

14.4 Email HTML injection — found and fixed. `email_utils.py` sends via Microsoft Graph's structured JSON `sendMail` call (not raw SMTP), so classic CRLF/header injection does not apply. However, `tariff_name` / `supplier` /rejection `reason` were interpolated **unescaped** into the HTML email bodies sent to admin and supplier inboxes on tariff submit/resubmit/approve/reject — all attacker-reachable via the tariff-submission form (schemas.py bounds `name` 's length but doesn't escape its content). This is the same bug class as the §8 stored-XSS findings, just landing in a mail client instead of a browser — a supplier could inject phishing links or tracking pixels into a legitimate CERA-branded email. Fixed with an `_esc()` (`html.escape`) helper applied at all 5 call sites touching an HTML body. Verified by mocking the Graph API call and injecting `<img src=x onerror=...>` / `<script>` payloads — captured payload showed properly escaped entities.

14.5 OTP brute-force — found and fixed. `verify_otp()` in `routers/auth.py` had no attempt limit independent of the password-login lockout (§4) — once past the password step, an attacker holding a `pending_token` could guess the 6-digit OTP (1,000,000 possibilities) unlimited times within its 10-minute validity. Fixed by reusing the existing `User.failed_attempts` / `locked_until` fields the password lockout already uses (no schema change needed, since a correct password already resets both to 0 immediately before OTP verification starts). Verified live: 2 wrong guesses → 401, a 3rd → 429 locked, and — critically — the actual correct code was also rejected while locked, proving the lockout blocks even an eventual correct guess, not just further wrong ones.

14.6 Dependency CVE scan — checked, clean. `pip-audit` against both `requirements.txt` and the actually-installed `tariff-api/.venv` environment found no known vulnerabilities in any resolved package (FastAPI, Starlette, SQLAlchemy, psycopg, PyJWT, bcrypt, msal, requests, Pydantic, email-validator, and transitive dependencies). Note: this surfaced that ad-hoc terminal commands (this review and presumably

earlier work) had been invoking the machine's global Python rather than the project's `.venv` — the venv itself was correctly set up and is now the one actually used for running/testing this app.

14.7 JWT storage — migrated from `sessionStorage` to an httpOnly cookie. See §2, §4, and §8 for the full description of the before/after design and the CSRF implication (`SameSite=Strict`). In short: `sessionStorage` was JS-readable, so any successful XSS could exfiltrate the live session token; an httpOnly cookie cannot be read by JavaScript, closing that theft vector even against XSS bugs not yet found. Verified live with a real Chromium browser via Playwright (not just an HTTP client — Python's `requests` / `http.cookiejar` doesn't implement the browser-specific "localhost counts as a secure context" exception needed to resend a `Secure` cookie over plain `http://` , so only a real browser test is authoritative here): cookie carries `HttpOnly` / `Secure` / `SameSite=Strict` with no `Max-Age` ; `document.cookie` never exposes the token; session persists across reload; logout clears the cookie server-side (a new `POST /api/auth/logout` endpoint, since JS cannot clear an httpOnly cookie itself) and does not silently re-authenticate; the two FormData document-upload paths that used to attach the token manually now work with zero explicit client-side auth code. 19 checks across both portals, zero console errors.

Deployment implication: the cookie's `Secure` flag means login requires HTTPS in a real deployment — this doesn't add a new requirement (HTTPS was already tracked below) but makes it load-bearing rather than just good practice; worth calling out explicitly to whoever handles the external deployment.

Known accepted limitation: opening the HTML files directly via `file://` instead of through the server no longer supports login (a cookie set by `http://localhost:8000` cannot be read back by a `file://` -origin page). Confirmed this doesn't affect the documented workflow — `start.bat` already serves everything same-origin via uvicorn.

---

15. Session hardening, accidental-overload protection & cross-portal access control (2026-07-15, later pass)

Prompted by a direct question — "are we protected against a buggy frontend retrying in a loop, or someone running a script that hammers the server by mistake?" — which led to fixing that gap and then, during live testing of the fix, surfaced two real access-control gaps between the admin and supplier portals. All items below were verified against the running app, not just read in code.

15.1 Per-IP rate limiting. No rate limiting of any kind existed previously. `slowapi` was tried first and found broken on this stack: FastAPI 0.139.0 / Starlette 1.3.1 wraps routers registered via `app.include_router()` in an internal `_IncludedRouter` object that slowapi's route-handler lookup doesn't recognize, so its limiter silently applied only to the one route declared directly on `app` (`/api/public/config`) and did nothing for `/api/auth/*` or `/api/admin/*` — confirmed empirically (a 3/min test limit tripped correctly on `/api/public/config` but never on `/api/auth/login` after 6+ requests). Replaced with a small self-contained sliding-window counter (`main.py` , keyed by client IP, no routing-library dependency, no third-party package) — 60 requests/minute per IP, returns `429` past that. In-memory only, correct for the current single-process deployment; would need a shared store (Redis) if ever scaled to multiple workers/instances. Rate-limit hits on `/api/auth/*` are also written into the existing `login_activity` table (`reason: "rate_limited"`) so they're visible in the admin panel's Login Activity

tab alongside lockouts, not just in server logs. Verified live: 65 rapid requests to `/api/auth/login` → first 60 succeed, remainder return `429`, each logged with email/IP.

15.2 Sliding session expiry with an absolute cap. See §4 (Token design) for the mechanism. Motivated by a direct question about whether shortening the flat token expiry (originally 8 hours) risked logging an active user out mid-work — it did, since no renewal mechanism existed at all. Fixed by making expiry slide on activity (1 hour, renewed every authenticated request) while capping total session lifetime at 12 hours from the original login (`session_start` claim, immune to renewal). Caught and fixed a real bug during testing: PyJWT only auto-converts the *registered* `iat` / `exp` / `nbf` claims from `datetime` — the custom `session_start` claim needed an explicit `.timestamp()` conversion or every request crashed. Verified live: chained requests show `session_start` unchanged while `iat` / `exp` advance; a hand-crafted 13-hour-old session was correctly rejected with `401` even though its own `exp` was still valid.

15.3 Identity re-hydration (`GET /api/auth/me`). See §4. Fixed a UX bug (not a security issue) where a valid session cookie in a fresh tab or after a reload never repopulated the frontend's cached identity (email/role in `sessionStorage`), so the Sign-out button and user badge silently never appeared even though the user was correctly authenticated.

15.4 Cross-portal role enforcement — found and fixed. Two frontend gaps let the wrong role load the wrong portal's UI: - `supplier.html` explicitly accepted `role === 'admin'` logins. - `admin.html` treated any `200` from `GET /api/admin/config` as proof of being an admin — but that endpoint just scopes tariffs down for supplier accounts rather than rejecting them (see §14.3), so a supplier's cookie on a fresh `admin.html` tab sailed straight through.

Both portals now explicitly verify the caller's role (via cached identity or `/api/auth/me`) before rendering their dashboard, and treat a mismatch exactly like "not authenticated" — showing the normal login screen — rather than auto-redirecting to the other portal (an earlier iteration of the fix did redirect; changed at the user's request so a wrong-role session doesn't get any information about where it *should* go). Wrong-role login/OTP attempts and any forced sign-out now show the same generic messages used for any other auth failure (`"Invalid email or password."` / `"Your session was terminated — please sign in again."`) — no mention of roles, portals, or account existence, to avoid giving an attacker probing the login form any information beyond "that didn't work."

Verified this was a UI exposure, not an authorization bypass, by attacking the API directly with a real supplier token, bypassing the frontend entirely: approving a tariff (`403 Admin access required`), deleting/creating/reading another supplier's tariff (`403 You can only modify your own tariffs` / `"You can only create tariffs for '<name>'"`), listing all portal users (`403`), and deleting a supplier (`403`) were all correctly rejected server-side — while the same account's own tariff remained fully accessible (`200`). Every mutating endpoint in `routers/admin.py` was also read end-to-end and confirmed to use `require_admin` or the `_assert_tariff_owner` / `is_supplier` ownership checks already documented in §4 and §14.3. The actual authorization boundary was correct throughout; what was broken was only which page's UI a given session was allowed to render.

15.5 Cross-tab session collision (`X-User-Role` header) — found and fixed. The session cookie is shared across every tab in a browser, not per-tab. Logging into a different role in a second tab silently overwrites the cookie everywhere, including an already-open tab on the other portal. In one direction (an admin tab whose cookie becomes a supplier's) this surfaces as `403`s on admin-only calls; in the other direction (a supplier tab whose cookie becomes an admin's) there is **no error at all** — admin sessions are

allowed everywhere — so status-code-based detection can't catch it, and the supplier tab would silently start receiving full unfiltered cross-supplier data.

Fixed with a new response header rather than reactive error handling: `main.py` middleware decodes whatever cookie is actually attached to each request and sets `X-User-Role` on every response. Both portals' existing global `fetch` interceptor (already used for the password-expiry gate) compares that header against the role the tab expects (`sessionStorage`) on every single response and forces a fresh login — same generic "session terminated" message as §15.4 — the moment they diverge, regardless of which endpoint triggered it. Verified live: a real admin token yields `X-User-Role: admin` , a real supplier token yields `X-User-Role: supplier` , an unauthenticated request has no such header (doesn't interfere with existing `401` handling).

Known accepted limitation, confirmed with the user rather than assumed: this does not allow simultaneous admin + supplier sessions in the same browser — that's a hard limit of any cookie-based session design (identical to why one browser can't hold two different Gmail accounts in two regular tabs), not something app code can fix without reintroducing the JS-readable-token XSS risk that §14.7 deliberately closed. The practical workaround for testing both roles at once is a separate browser profile or a private/incognito window.

---

16. Functional/data-integrity work, 2026-07-17 to 2026-07-20 (not a security review pass)

Five days of ordinary feature and bugfix work followed §15, none of it prompted by a security review, but three items are relevant to this document and are recorded here so the doc stays an accurate description of the running system.

16.1 Supplier↔user linkage replaced with a real foreign key. Previously `User.supplier_name` (and `MessageRead.supplier_name`) were plain strings that had to exactly match `Supplier.name` , with no FK enforcing the relationship — renaming a supplier in the admin panel silently orphaned any user row still holding the old name, with no error. This wasn't a hypothetical: renaming "EAC Supply" cut off a real supplier user (`cera.progress@gmail.com`) from all 6 of their tariffs. Fixed by adding a real `supplier_id` FK (`ForeignKey("suppliers.id")`) to both tables, backfilled via `scripts/migrate_supplier_fk.sql` (verified zero NULLs before the old string columns were dropped via `scripts/drop_supplier_name_columns.sql`). `CurrentUser.supplier_name` is now resolved live on every request via `auth.resolve_supplier_name()` (a `db.get(Supplier, user.supplier_id)` lookup) rather than read from a stored column, so a rename is reflected instantly everywhere with nothing left to cascade. `Tariff.supplier` and `Message` / `ScheduledNotification.target_suppliers` were deliberately left as plain strings (a bigger normalization effort, and not where the actual bug lived); `update_supplier()` 's existing rename-cascade logic now also covers `ScheduledNotification.target_suppliers` , which the original cascade had missed.

16.2 Scheduled local database backup. §17 previously listed "no managed backup" as an open risk with zero mitigation in place. There is now a **daily local** `pg_dump` (`tariff-api/scripts/backup_db.ps1` , Windows Scheduled Task, 10:00 daily) producing a custom-format (`-Fc`) archive with 14-day daily + 8-week weekly rolling retention and automatic pruning, written to a OneDrive-synced folder (giving it off-

machine redundancy without a manual step). Credentials are read from `%APPDATA%\postgresql\pgpass.conf`, not embedded in the script. Verified end-to-end: both a manual run and an actual Scheduled Task trigger produced an archive that `pg_restore -l` confirmed contains all 101 database objects. The restore procedure (overwrite-in-place vs. restore-to-a-throwaway-database-to-inspect-first) is documented in `ADMIN_MANUAL.md` §15.5. This is explicitly **temporary insurance for the current dev-only Postgres instance**, not a replacement for real managed backups once migrated to Azure Database for PostgreSQL (Flexible Server's built-in automated backups supersede this local script at that point) — §17 item 4 below is updated to reflect partial mitigation, not closure.

16.3 Changelog data-loss bug found and fixed — worth the external reviewer's attention. The `changelog` table is meant to be an append-only audit trail (§3), but the legacy bulk "Save & publish" flow (`importer.py`'s `import_cfg()`) was deleting and rebuilding it from whatever snapshot the browser happened to hold in memory at the time — **this had already silently wiped the production changelog table before the bug was found**, with no backup in place at the time to recover it (predates §16.2). Fixed by making every admin section (suppliers, load profiles, notices, DAM data — tariffs and regulated charges already worked this way) save itself immediately via its own REST endpoint instead of staging changes for a bulk publish step, and by removing `import_cfg()`'s changelog-clearing behavior entirely; "Save & publish" is now an optional manual full-resync checkpoint rather than the only way changes reached the database. A related orphaning bug was fixed alongside it: "overwrite profile by name" previously generated a fresh random id every time, silently breaking any tariff's `profileId` reference to the old row — it now reuses the existing profile's id. Also: the changelog UI now resolves `profileId` changes to the profile's current name and maps raw camelCase field keys to human-readable labels, and a "Clear log" button was removed (no longer appropriate for an append-only audit trail). `ADMIN_MANUAL.md` (now v1.4) documents the new per-section save behavior.

Why this belongs in a security-facing document: an append-only audit trail silently losing data on a routine admin action, with no backup to recover it, is exactly the kind of integrity gap an external assessment would flag — worth disclosing proactively rather than letting it surface as a "finding." It is now fixed and the backup in §16.2 provides a recovery path if a similar bug recurs.

---

17. Known outstanding risks (pre-production checklist)

These are tracked items, not yet remediated, in priority order for an external certification review:

1. **Secrets in plaintext** `.env` (§5) — highest priority, easiest finding for any reviewer.
2. **Azure** `Mail.Send` **application permission not yet scoped** via Exchange Application Access Policy (§5) — requires a Global/Exchange admin.
3. **No HTTPS/reverse proxy** in the current setup — must sit behind TLS termination before external exposure. Now load-bearing, not just best practice: the session cookie's `Secure` flag (§14.7) means login will silently fail without it.
4. **Dev-only PostgreSQL instance** — user-owned, non-service process, no TLS/HA. **Partially mitigated 2026-07-17 (§16.2)**: a daily local `pg_dump` with rolling retention now exists, so the data is no longer unrecoverable — but this is temporary insurance, not a substitute for a real managed Postgres (e.g. Azure Database for PostgreSQL) with proper TLS/HA before production.

5. **No Alembic migrations** — schema changes are hand-rolled `ALTER TABLE` statements at startup; works but has no rollback path.
6. **Minor audit-trail gaps**: OTP-toggle-per-user and document-upload-triggered-resubmission are not written to `changelog` (only explicit status-patch actions are).
7. Document-upload feature has no malware/content scanning (acceptable today given admin-gated approval, worth reassessing if exposure changes).
8. **Local git history only, no remote** — adequate for a single-developer project, but there's no off-machine backup of commit history.

Two items from the original version of this list are now closed and removed from tracking: stored XSS on the public customer page (§8, fixed 2026-07-14) and the absence of version control (§1, a local repository now exists).

An external IT company is expected to perform a combined functional/UAT walkthrough and OWASP-style security assessment before go-live; the items above are the ones most likely to surface in that assessment and should be prioritized accordingly.

---

18. Summary assessment

The backend shows a consistent, deliberate security design for its size, strengthened further by the 2026-07-14/15 hardening passes: cookie-based session auth with `httpOnly` / `Secure` / `SameSite=Strict` (immune to both XSS-based token theft and CSRF), sliding expiry with an absolute session cap (§15.2), server-side ownership checks on every mutating endpoint (including the changelog/supplier-list gaps closed in §14.3, and re-verified end-to-end during the §15.4 cross-portal review), account lockout with audit logging extended to cover OTP guessing as well as password attempts (§14.5), password history/expiry/reuse enforcement, session invalidation on password change, Pydantic-enforced input validation matching the frontend's actual value sets (§13.3), a Content-Security-Policy header, DOMPurify sanitization on the one deliberate raw-HTML rendering path, consistent output escaping across all 116 audited `innerHTML` sinks, and per-IP rate limiting against accidental overload (§15.1). SQL injection and dependency CVEs were checked and found clean; buffer overflow is not an applicable risk for this stack. The two cross-portal UI gaps found in §15.4/§15.5 were UX/access-routing issues, not authorization bypasses — the underlying permission checks held throughout, confirmed by direct API attack attempts, not just code reading. The remaining gaps are entirely operational/infrastructure (secrets storage, TLS, managed DB, Alembic migrations) rather than architectural — none of the previously-open code-level findings remain outstanding.